



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

课程报告

开课学期: 2026 夏季

课程名称: 计算机设计与实践

项目名称: 支持 mini RV 的 SoC 设计

项目类型: 综合设计型

课程学时: 48 地点: T2507

学生班级: 计算机 (拔尖) 2 班、
计算机 (拔尖) 2 班

学生学号: 2024311668、2024311669

学生姓名: 高呈歌、郑久文

评阅教师: _____

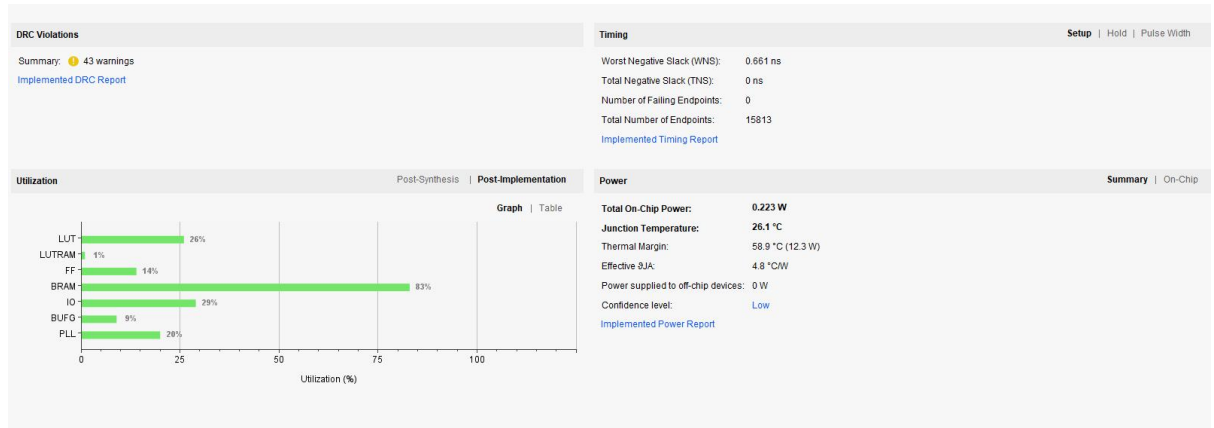
报告成绩: _____

实验与创新实践教育中心制

2026 年 7 月

注：本报告的各部分如果页数不够，请同学们自行扩页。原则上一定要把报告写详细，能说明清楚设计实现的主要逻辑、设计的成果、特色和过程等。设计报告将是评定每个人成绩的重要组成部分（设计内容及报告写作都作为评分依据）。

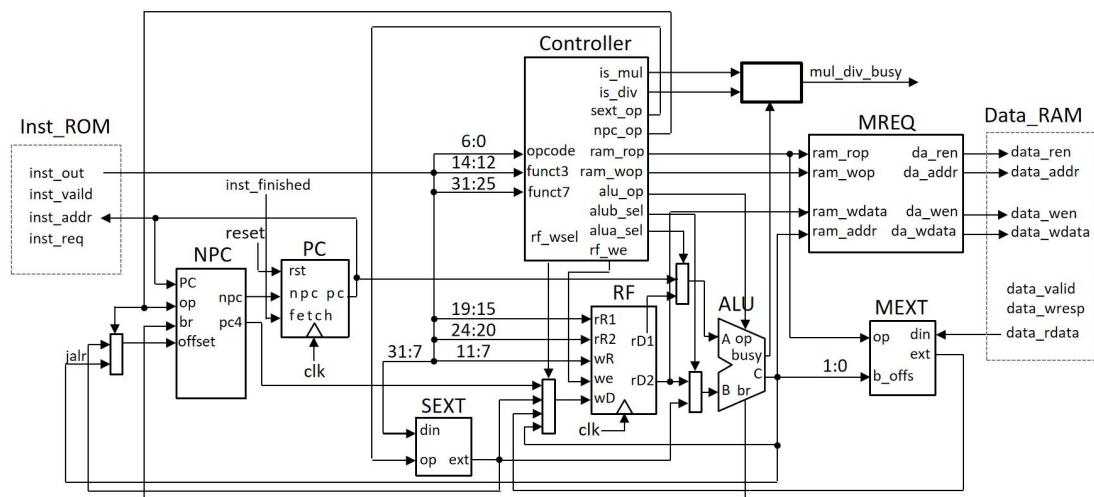
设计概述			
组号	姓名	完成的指令	完成的其他内容
26	高呈歌	SLL、SRL、SRLI、SRA、SRAI、ADD、SUB、AUIPC、XOR、XORI、LB、LBU、LH、LHU、SW、SB、SH、JALR，共 18 条 miniRV A 组指令。	完成 miniRV A 组指令的数据通路表、控制信号表及单周期 CPU 代码实现，A 组指令的 Basic Trace 测试。在完整单周期 CPU 基础上设计 IF/ID、ID/EX、EX/MEM、MEM/WB 流水寄存器；实现默认不跳转的静态分支预测、错误路径清空、数据冒险检测、流水线暂停及数据前递机制、Load-Use 冒险、乘除法多周期运算和访存等待的暂停控制，流水线 CPU 的 AXI Trace 调试、CoreMark 程序下板运行。
	郑久文	MUL、MULH、MULHU、DIV、DIVU、REM、REMU、SLT、SLTI、SLTU、SLTIU、OR、AND、ANDI、BLT、BGE、BLTU、BGEU，共 18 条 miniRV B 组指令。	完成 miniRV B 组指令的数据通路表、控制信号表及单周期 CPU 代码实现，并完成 B 组指令的 Basic Trace 测试；参与完整单周期 CPU 的数据通路设计、代码整合与调试。在完整单周期 CPU 基础上完成 SoC 扩展，集成 ICACHE、DCACHE 存储阵列，设计基于状态机的 AXI 总线控制器、总线桥、主存接口及 Uncached 访问路径；完成拨码开关、LED、数码管、UART 和计时器等存储器映射外设的 I/O 接口电路及 C 语言测试程序；完成单周期 SoC 的 AXI Trace 测试和 C_TEST 0~2 下板测试，并参与流水线 CPU 与 SoC 整合。
设计的主要特色（除基本要求以外的设计）			
本设计在完成课程基本要求的基础上，对部分实现结构进行了调整。流水线取指级增加了双入口指令队列，用于暂存流水线暂停期间已经返回的指令，并在分支跳转时清除错误路径指令和未返回的取指响应。针对板载时钟与 PLL 系统时钟之间曾出现的复位时序问题，顶层采用异步复位、同步释放的两级复位同步器。外设部分没有完全采用指导书中的 AXI GPIO 和协议转换 IP 核，而是通过自定义 AXI 地址路由及外设 RTL 模块完成主存和外设的地址分流。			
流水线 SoC 在实现后（Post Implementation）的资源使用、功耗数据截图			



1 单周期 CPU 设计与实现

1.1 单周期 CPU 数据通路设计

要求：贴出完整的单周期数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。



1. Inst_ROM (指令存储器)

用于存放程序指令。根据 PC 给出的取指地址读取对应的 32 位机器码，并通过有效信号通知 CPU 指令已返回。

2. PC (程序计数器)

用于保存当前指令地址，并向指令存储器发出取指请求。每条指令执行完成后，PC 更新为 NPC 模块计算出的下一条指令地址。

3. NPC (下一 PC 生成模块)

根据当前 PC、立即数偏移量、分支判断结果和控制信号，计算下一条指令地址。普通指令选择 PC+4，跳转或分支指令选择相应的目标地址。

4. Controller (控制器)

对指令中的 opcode、funct3 和 funct7 字段进行译码，判断当前指令类型，并产生寄存器写使能、ALU 操作、立即数扩展、访存方式和下一 PC 选择等控制信号。

5. RF (寄存器堆)

用于保存通用寄存器数据。根据指令中的源寄存器编号同时读出两个操作数，并在写使能有效时将运算结果或访存数据写入目标寄存器。

6. SEXT (立即数扩展模块)

根据不同指令格式提取指令中的立即数字段，并将其进行符号扩展或重新拼接，生成 32 位立即数供 ALU、访存和跳转计算使用。

7. ALU (算术逻辑单元)

完成加、减、移位、逻辑运算和比较等操作。对于访存指令，ALU 还负责计算访存地址；对于分支指令，ALU 输出分支判断结果。

8. 乘法单元

用于完成乘法和除法相关指令。运算过程中通过 mul_div_busy 信号表示当前单元是否仍在工作，避免 CPU 提前更新执行状态。

9. MREQ (数据访存请求模块)

将 CPU 内部的访存操作转换为数据存储器接口信号，生成读使能、写使能、访存地址和写入数据，并接收数据存储器返回的有效信号或写响应信号。

10. MEXT (访存数据扩展模块)

对 Load 指令从存储器读回的数据进行处理。根据访问类型和地址低位，选取对应的字节、半字或字，并完成符号扩展或零扩展。

11. Data_RAM (数据存储器)

用于保存程序运行过程中使用的数据。根据地址和读写使能完成数据读取或写入，并向 CPU 返回读取数据、数据有效信号和写响应信号。

12. 多路选择器

根据控制信号在多组输入数据中选择正确的数据通路，主要用于选择下一 PC、ALU 第二操作数以及寄存器写回数据等。

总体上，控制器负责产生控制信号，其余模块按照控制信号完成取指、译码、运算、访存和写回，使一条指令在一个时钟周期内完成主要的数据处理过程。

1.2 单周期 CPU 详细设计

要求：以表格的形式列出各个部件的接口信号、位宽、功能描述等，并结合图、表、核心代码等形象化工具和手段，详细描述各个部件的关键实现。

1. PC 程序计数器模块

接口信号	方向	位宽	功能描述
clk	输入	1 bit	CPU 时钟信号
rst	输入	1 bit	高电平复位信号
npc	输入	32 bit	NPC 模块计算出的下一条指令地址
fetch	输入	1 bit	当前指令执行完成、允许更新 PC 的标志
pc	输出	32 bit	当前指令地址

PC 模块用于保存当前正在执行指令的地址。复位时将 PC 清零，使 CPU 从地址 0 开始执行；正常情况下，只有 fetch 有效时才把 npc 写入 PC，否则保持原来的地址。该设计可以在访存或乘除法尚未完成时暂停 PC 更新，避免提前执行下一条指令。

```
always @(posedge clk or posedge rst) begin
    if (rst) pc <= 32'h0;
    else    pc <= fetch ? npc : pc;
end
```

2. NPC 下一指令地址生成模块

接口信号	方向	位宽	功能描述
op	输入	2 bit	下一 PC 选择控制信号
pc	输入	32 bit	当前指令地址
offset	输入	32 bit	分支或跳转目标偏移量
br	输入	1 bit	ALU 输出的分支条件判断结果
npc	输出	32 bit	下一条指令地址
pc4	输出	32 bit	当前 PC 加 4 的结果

NPC 模块根据 npc_op 选择下一条指令地址。普通指令选择 PC+4；JAL 使用 PC 加扩展后的 J 型立即数；条件分支在 br 为 1 时选择 PC 加偏移量，否则仍选择 PC+4；JALR 使用 ALU 算出的目标地址，并将最低位清零以满足 RISC-V 地址对齐要求。核心代码：

```

assign pc4 = pc + 32'h4;

always @(*) begin
    case (op)
        `NPC_PC4 : npc = pc4;
        `NPC_JALR: npc = {offset[31:1], 1'b0};
        `NPC_BRA : npc = br ? pc + offset : pc4;
        `NPC_JMP : npc = pc + offset;
        default  : npc = pc4;
    endcase
end

```

3. Controller 控制器模块

接口信号	方向	位宽	功能描述
opcode	输入	7 bit	指令操作码，对应 inst[6:0]
funct3	输入	3 bit	功能字段，对应 inst[14:12]
funct7	输入	7 bit	功能字段，对应 inst[31:25]
npc_op	输出	2 bit	NPC 地址选择控制信号
sext_op	输出	3 bit	立即数扩展类型
alua_sel	输出	1 bit	ALU 操作数 A 选择：RS1 或 PC
alub_sel	输出	1 bit	ALU 操作数 B 选择：RS2 或立即数
alu_op	输出	5 bit	ALU 运算类型
is_mul	输出	1 bit	乘法类指令标志
is_div	输出	1 bit	除法或求余类指令标志
ram_r_op	输出	3 bit	Load 读数据类型及扩展方式
ram_w_op	输出	4 bit	Store 写使能类型
rf_we	输出	1 bit	寄存器写使能
rf_wsel	输出	2 bit	寄存器写回数据来源选择

Controller 是数据通路的控制核心。模块首先根据 opcode、funct3 和 funct7 判断当前指令，再把指令类别转换为 NPC、SEXT、ALU、访存和寄存器堆需要的控制信号。例如 ADD 和 SUB 的 opcode、funct3 相同，需要继续根据 funct7 进行区分；Load 和 Store 则通过 funct3 判断字、半字或字节访问。

```

wire ADD = (opcode == 7'b0110011) &&
            (funct3 == 3'b000) && (funct7 == 7'b0000000);
wire SUB = (opcode == 7'b0110011) &&
            (funct3 == 3'b000) && (funct7 == 7'b0100000);
wire LB  = (opcode == 7'b0000011) && (funct3 == 3'b000);
wire SW  = (opcode == 7'b0100011) && (funct3 == 3'b010);

```

控制信号：

控制信号	主要取值	作用
npc_op	PC4、JALR、BRA、JMP	选择顺序执行、间接跳转、条件分支或直接跳转
sext_op	I、S、B、U、J	选择不同指令格式的立即数拼接方式
alu_op	ADD、SUB、SLL、SRL、SRA等	控制 ALU 执行具体运算或比较
alua_sel	RS1 或 PC	选择 ALU 第一个操作数
alub_sel	RS2 或 EXT	选择 ALU 第二个操作数
ram_r_op	N、W、B、BU、H、HU	控制 Load 访问类型和读回数据扩展
ram_w_op	N、B、H、W	控制 Store 按字节、半字或字写入
rf_wsel	ALU、RAM、PC4、EXT	选择写回寄存器的数据来源

核心代码：

```

assign npc_op = {2{NPC_OP_PC4}} & `NPC_PC4
               | {2{NPC_OP_JALR}} & `NPC_JALR
               | {2{NPC_OP_BRA}} & `NPC_BRA
               | {2{NPC_OP_JMP}} & `NPC_JMP;

assign rf_wsel = {2{WB_OP_ALU}} & `WB_ALU
               | {2{WB_OP_RAM}} & `WB_RAM
               | {2{WB_OP_PC4}} & `WB_PC4
               | {2{WB_OP_EXT}} & `WB_EXT;

```


4. RF 寄存器堆模块

接口信号	方向	位宽	功能描述
clk	输入	1 bit	寄存器写入时钟
rR1	输入	5 bit	源寄存器 1 编号
rR2	输入	5 bit	源寄存器 2 编号
we	输入	1 bit	寄存器写使能
wR	输入	5 bit	目标寄存器编号
wD	输入	32 bit	写回数据
rD1	输出	32 bit	源寄存器 1 读出数据
rD2	输出	32 bit	源寄存器 2 读出数据

RF 模块实现 32 个通用寄存器,其中 x0 固定为 0,因此代码只实际存储 x1 到 x31。两个读端口采用组合读,可在同一周期同时读出两个源操作数;写端口采用时钟上升沿写入。写入时还要判断目标寄存器不是 x0,防止破坏零寄存器的固定值。

```
reg [31:0] regs [1:31];

assign rD1 = (rR1 == 0) ? 32'h0 : regs[rR1];
assign rD2 = (rR2 == 0) ? 32'h0 : regs[rR2];

always @(posedge clk) begin
    if (we && (wR != 5'h0)) regs[wR] <= wD;
end
```

5. SEXT 立即数扩展模块

接口信号	方向	位宽	功能描述
op	输入	3 bit	立即数格式选择信号
imm	输入	25 bit	指令 inst[31:7]中的立即数字段
ext	输出	32 bit	拼接并扩展后的 32 位立即数

SEXT 模块根据指令格式生成 32 位立即数。I 型用于立即数运算、Load 和 JALR; S 型用于 Store; B 型用于条件分支; U 型用于 LUI 和 AUIPC; J 型用于 JAL。I、S、B 和 J 型立即数需要进行符号扩展, B 型和 J 型最低位补 0, 表示跳转地址按 2 字节对齐。

```

case (op)
  `EXT_I: ext = {{20{imm[31]}}, imm[31:20]};
  `EXT_S: ext = {{20{imm[31]}}, imm[31:25], imm[11:7]};
  `EXT_B: ext = {{19{imm[31]}}, imm[31], imm[7],
                imm[30:25], imm[11:8], 1'b0};
  `EXT_U: ext = {imm[31:12], 12'h0};
  `EXT_J: ext = {{11{imm[31]}}, imm[31], imm[19:12],
                imm[20], imm[30:21], 1'b0};
  default: ext = 32'h0;
endcase

```

6. ALU 算术逻辑与乘除法模块

接口信号	方向	位宽	功能描述
rst	输入	1 bit	乘除法状态复位信号
clk	输入	1 bit	乘除法迭代运算时钟
op	输入	5 bit	运算或比较类型
a	输入	32 bit	操作数 A
b	输入	32 bit	操作数 B
c	输出	32 bit	算术逻辑运算结果
br	输出	1 bit	条件分支判断结果
busy	输出	1 bit	乘除法运算忙信号

ALU 模块完成加减、逻辑、移位、大小比较、分支条件判断以及乘除法运算。普通算术逻辑运算为组合逻辑，输入变化后直接得到结果；分支指令通过 br 输出比较结果；乘除法由独立的迭代模块完成，在运算过程中 busy 保持为 1，CPU 等待 busy 变为 0 后再写回结果。

```

case (op_now)
  `ALU_ADD : c = a_now + b_now;
  `ALU_SUB : c = a_now - b_now;
  `ALU_AND : c = a_now & b_now;
  `ALU_OR  : c = a_now | b_now;
  `ALU_XOR : c = a_now ^ b_now;
  `ALU_SLL : c = a_now << b_now[4:0];
  `ALU_SRL : c = a_now >> b_now[4:0];
  `ALU_SRA : c = $signed(a_now) >>> b_now[4:0];
  `ALU_SLT : c = ($signed(a_now) < $signed(b_now));
  `ALU_SLTU: c = (a_now < b_now);
  default  : c = 32'h0;
endcase

```

分支判断

```

case (op)
  `ALU_EQ : br = (a == b);
  `ALU_NE : br = (a != b);
  `ALU_LT : br = ($signed(a) < $signed(b));
  `ALU_GE : br = ($signed(a) >= $signed(b));
  `ALU_LTU: br = (a < b);
  `ALU_GEU: br = (a >= b);
  default : br = 1'b0;
endcase

```

乘法器采用移位累加方法，每个周期检查乘数最低位，决定是否将被乘数累加到部分积中，然后被乘数左移、乘数右移，完成 WIDTH 次迭代后输出结果。除法器采用恢复余数法，每个周期完成一次移位、比较和减法，最终得到商和余数。ALU 还对除数为 0 以及有符号除法溢出进行了单独处理。

接口信号	方向	位宽	功能描述
x、y	输入	WIDTH bit	被乘数/乘数或被除数/除数
start	输入	1 bit	启动一次运算
z	输出	2×WIDTH 或 WIDTH bit	乘积或商
r	输出	WIDTH bit	除法余数，仅 divider 使用
busy	输出	1 bit	运算进行中标志

clk	输入	1 bit	迭代运算时钟
rst	输入	1 bit	运算状态复位

7.MREQ 访存请求生成模块

接口信号	方向	位宽	功能描述
ram_addr	输入	32 bit	ALU 计算出的字节地址
ram_rop	输入	3 bit	Load 访问类型
da_ren	输出	4 bit	数据存储器读使能
da_addr	输出	32 bit	送往数据存储器的地址
ram_wop	输入	4 bit	Store 访问类型
ram_wdata	输入	32 bit	待写入数据，来自 RF 第二读端口
da_wen	输出	4 bit	按字节写使能
da_wdata	输出	32 bit	对齐后的写数据

MREQ 模块负责把 CPU 内部的 Load 和 Store 操作转换为数据存储器接口信号。地址最低两位表示访问数据在 32 位存储字中的字节位置。对于 SB 和 SH，模块根据地址偏移产生对应的字节写使能，并将有效数据移动到正确的字节位置；SW 要求地址按 4 字节对齐。Load 请求统一读取一个 32 位存储字，随后由 MEXT 选取需要的字节或半字。

Store 写使能与数据对齐示例

```

case (ram_wop)
  `RAM_WE_B: begin
    case (ram_addr[1:0])
      2'b00: begin da_wen=4'b0001; da_wdata=ram_wdata; end
      2'b01: begin da_wen=4'b0010; da_wdata={16'h0,ram_wdata[7:0],8'h0}; end
      2'b10: begin da_wen=4'b0100; da_wdata={8'h0,ram_wdata[7:0],16'h0}; end
      2'b11: begin da_wen=4'b1000; da_wdata={ram_wdata[7:0],24'h0}; end
    endcase
  end
  `RAM_WE_H: ...
  `RAM_WE_W: if (ram_addr[1:0] == 2'b00) da_wen = 4'b1111;
endcase

```

Load 读请求生成示例

```

if (ram_rop != `RAM_EXT_N) begin
    case (ram_rop)
        `RAM_EXT_B, `RAM_EXT_BU: da_ren = 4'hF;
        `RAM_EXT_H, `RAM_EXT_HU:
            da_ren = (ram_addr[0] == 1'b0) ? 4'hF : 4'h0;
        default:
            da_ren = (ram_addr[1:0] == 2'b00) ? 4'hF : 4'h0;
    endcase
end else da_ren = 4'h0;

```

8.MEXT 访存数据扩展模块

接口信号	方向	位宽	功能描述
op	输入	3 bit	Load 数据扩展类型
din	输入	32 bit	数据存储器返回的 32 位数据
byte_offs	输入	2 bit	访存地址最低两位
ext	输出	32 bit	选取并扩展后的写回数据

MEXT 模块先根据 byte_offs 把目标字节或半字移动到数据低位，再按照 Load 类型进行符号扩展或零扩展。LB 和 LH 使用最高有效位进行符号扩展，LBU 和 LHU 在高位补 0，LW 直接保留完整 32 位数据。

MEXT 模块核心代码

```

case (byte_offs)
    2'b01 : real_din = { 8'h0, din[31: 8]};
    2'b10 : real_din = {16'h0, din[31:16]};
    2'b11 : real_din = {24'h0, din[31:24]};
    default: real_din = din;
endcase

case (op)
    `RAM_EXT_B : ext = {{24{real_din[7]}}, real_din[7:0]};
    `RAM_EXT_BU: ext = {24'h0, real_din[7:0]};
    `RAM_EXT_H : ext = {{16{real_din[15]}}, real_din[15:0]};
    `RAM_EXT_HU: ext = {16'h0, real_din[15:0]};
    default    : ext = real_din;
endcase

```

9. Inst_ROM 指令存储器接口模块

接口信号	方向	位宽	功能描述
cpu_clk	输入	1 bit	CPU 时钟信号
cpu_rst	输入	1 bit	高电平复位信号
inst_rreq	输入	1 bit	CPU 发出的取指请求
inst_addr	输入	32 bit	取指字节地址
inst_valid	输出	1 bit	返回指令有效信号
inst_out	输出	32 bit	返回的 32 位机器码

Inst_ROM 模块对 Block Memory IP 核进行封装。CPU 给出字节地址，但 IROM 按 32 位字寻址，因此使用 inst_addr[31:2] 作为 IP 核地址。取指请求在下一个时钟周期形成 inst_valid，表示 inst_out 中的机器码有效。

Inst_ROM 模块核心代码

```
always @(posedge cpu_clk or posedge cpu_rst) begin
    inst_valid <= cpu_rst ? 1'b0 : inst_rreq;
end

IROM U_irom (
    .clka (cpu_clk),
    .addra (inst_addr[31:2]),
    .douta (inst_out)
);
```

10. Data_RAM 数据存储器接口模块

接口信号	方向	位宽	功能描述
cpu_clk	输入	1 bit	CPU 时钟信号
cpu_rst	输入	1 bit	高电平复位信号
data_ren	输入	4 bit	读使能
data_addr	输入	32 bit	读写字节地址
data_valid	输出	1 bit	读数据有效信号
data_rdata	输出	32 bit	返回的 32 位数据

data_wen	输入	4 bit	按字节写使能
data_wdata	输入	32 bit	待写入数据
data_wresp	输出	1 bit	写操作完成响应

Data_RAM 模块同样封装 Block Memory IP 核。地址使用 data_addr[31:2]按字访问，data_wen 的 4 个比特分别控制 32 位数据中的 4 个字节。检测到读使能后在下一周期拉高 data_valid；检测到写使能后拉高 data_wresp，用于通知 CPU 当前访存操作已经完成。

Data_RAM 模块核心代码

```
always @(posedge cpu_clk or posedge cpu_rst) begin
    data_valid <= cpu_rst ? 1'b0 : |data_ren;
    data_wresp <= cpu_rst ? 1'b0 : |data_wen;
end

DRAM U_dram (
    .clka (cpu_clk),
    .addra (data_addr[31:2]),
    .douta (data_rdata),
    .wea (data_wen),
    .dina (data_wdata)
);
```

11. cpu_core 顶层连接与指令完成控制

接口信号	方向	位宽	功能描述
cpu_rst	输入	1 bit	CPU 高电平复位
cpu_clk	输入	1 bit	CPU 工作时钟
ifetch_req	输出	1 bit	取指请求
ifetch_addr	输出	32 bit	取指地址
ifetch_valid	输入	1 bit	返回指令有效
ifetch_inst	输入	32 bit	返回的机器码
daccess_ren	输出	4 bit	数据读使能
daccess_addr	输出	32 bit	数据访问地址
daccess_rvalid	输入	1 bit	读数据有效

daccess_rdata	输入	32 bit	读回数据
daccess_wen	输出	4 bit	数据写使能
daccess_wdata	输出	32 bit	写数据
daccess_wresp	输入	1 bit	写操作完成响应

cpu_core 负责实例化并连接各功能模块。取回的指令只在 ifetch_valid 有效时使用，否则送入 NOP 指令，避免存储器输出变化导致误执行。ALU 第一个操作数由 alua_sel 在 PC 和 rf_rd1 之间选择，第二个操作数由 alub_sel 在扩展立即数和 rf_rd2 之间选择。

主要数据选择逻辑

```
assign inst = ifetch_valid ? ifetch_inst : 32'h00000013; // NOP
assign alu_a = alua_sel ? pc : rf_rd1;
assign alu_b = alub_sel ? ext : rf_rd2;
assign npc_offset = (npc_op == `NPC_JALR) ? alu_c : ext;
```

写回阶段根据指令类型选择数据来源：普通运算写回 ALU 结果，JAL/JALR 写回 PC+4，LUI 写回扩展立即数，Load 写回 MEXT 处理后的存储器数据。rf_we1 是最终写使能，保证不同类型指令只在结果真正有效时写入寄存器。

寄存器写回控制

```
assign rf_we1 = ld_st_flag & daccess_rvalid |
               mul_div_flag & !mul_div_busy |
               ifetch_valid & rf_we & !is_ld_st & !is_mul_div;

always @(*) begin
    casex ({ld_st_flag, rf_wsel})
        {1'b0, `WB_ALU}: rf_wD = alu_c;
        {1'b0, `WB_PC4}: rf_wD = pc4;
        {1'b0, `WB_EXT}: rf_wD = ext;
        {1'b1, 2'b??}: rf_wD = ram_ext;
        default        : rf_wD = 32'h0;
    endcase
end
```

Load、Store 和乘除法不能在取回指令的同一周期结束，因此代码设置 ld_st_flag 和 mul_div_flag 保存执行状态，并缓存目标寄存器、ALU 地址以及 Load 类型。访存操作收到 daccess_rvalid 或 daccess_wresp 后结束；乘除法在 busy 拉低后结束。只有 inst_finished 有效时 PC 才更新，并在下一周期发出新的取指请求。


```
assign ld_st_done = daccess_rvalid | daccess_wresp;

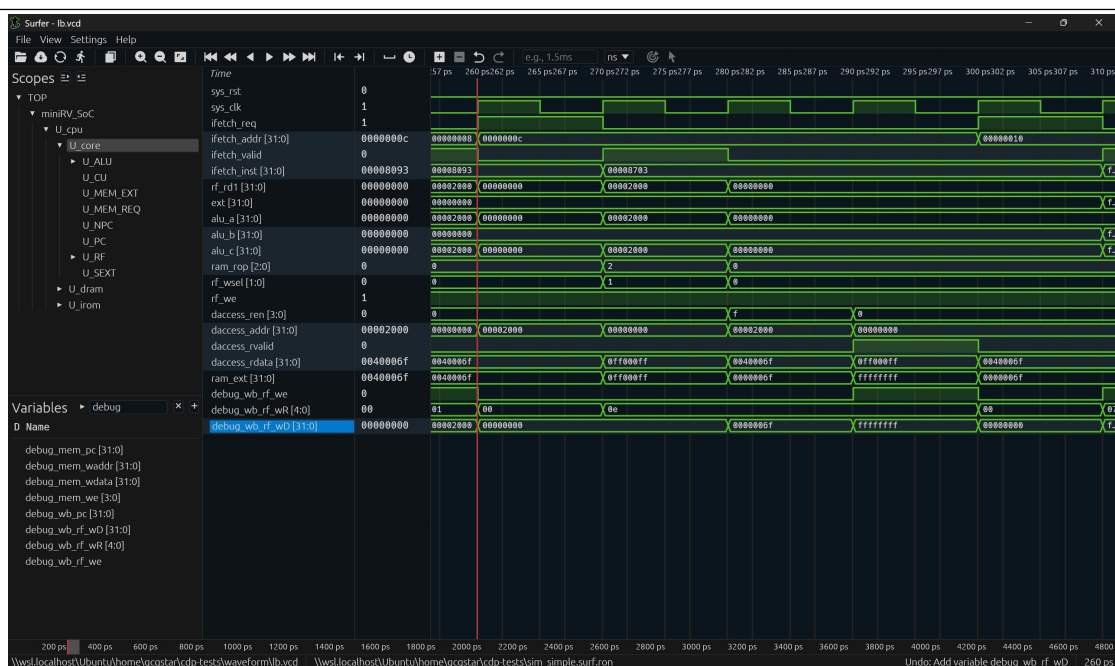
assign inst_finished =
    ld_st_flag  & ld_st_done      |
    mul_div_flag & !mul_div_busy  |
    ifetch_valid & !is_ld_st & !is_mul_div;

assign ifetch_req = first_req | inst_finished_r;
```

1.3 单周期 CPU 仿真分析

要求：在半字/字节访存指令、乘除法指令中分别任选 1 条指令，结合波形截图进行分析。截图需清晰，且需要包含信号名和指令执行所需的关键信号；此外，

要结合波形具体时刻具体信号值来分析指令执行过程。



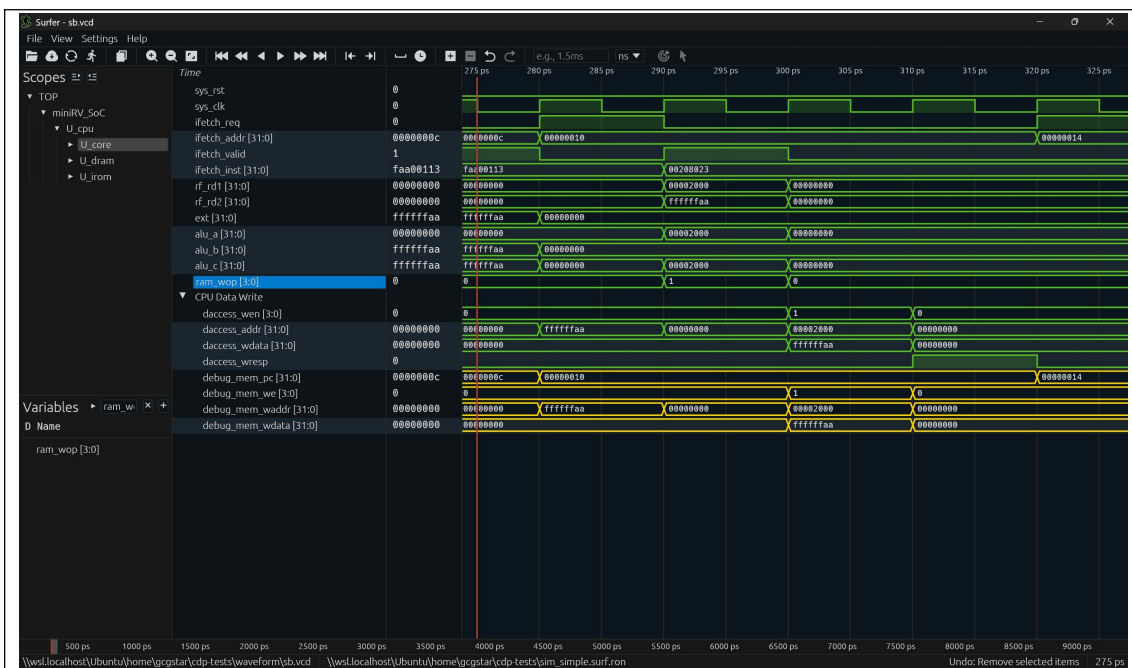
lb 指令仿真分析：

选取指令地址为 0x0000000c 的 lb x14, 0(x1) 进行分析，其机器码为 0x00008703。

在 260 ps 时，ifetch_req=1，同时 ifetch_addr=0x0000000c，说明 CPU 向指令存储器发出了取指请求。在 270 ps 时，ifetch_valid=1，ifetch_inst=0x00008703，表示该条 lb 指令已经返回并开始执行。此时源寄存器值 rf_rd1=0x00002000，立即数扩展结果 ext=0x00000000，ALU 计算得到访存地址 alu_c=0x00002000。控制信号 ram_rop=2 表示进行有符号字节读取，rf_wsel=1 表示选择访存结果写回寄存器。

在 280 ps 时，daccess_ren=4'hF，daccess_addr=0x00002000，CPU 向数据存储器发出了读请求。在 290 ps 时，daccess_rvalid=1，存储器返回数据 daccess_rdata=0x0ff000ff。由于访问地址的低两位为 00，因此选取最低字节 0xff。lb 为有符号字节读取，故将 0xff 符号扩展为 ram_ext=0xffffffff。

从图中可以看出，同一时刻 debug_wb_rf_we=1、debug_wb_rf_wR=0x0e、debug_wb_rf_wD=0xffffffff，说明 CPU 最终将 0xffffffff 写入寄存器 x14。在 300 ps 时，ifetch_addr 更新为 0x00000010，CPU 开始读取下一条指令，说明该条 lb 指令执行完成。仿真结果与 lb 指令的预期功能一致。



sb 指令仿真分析:

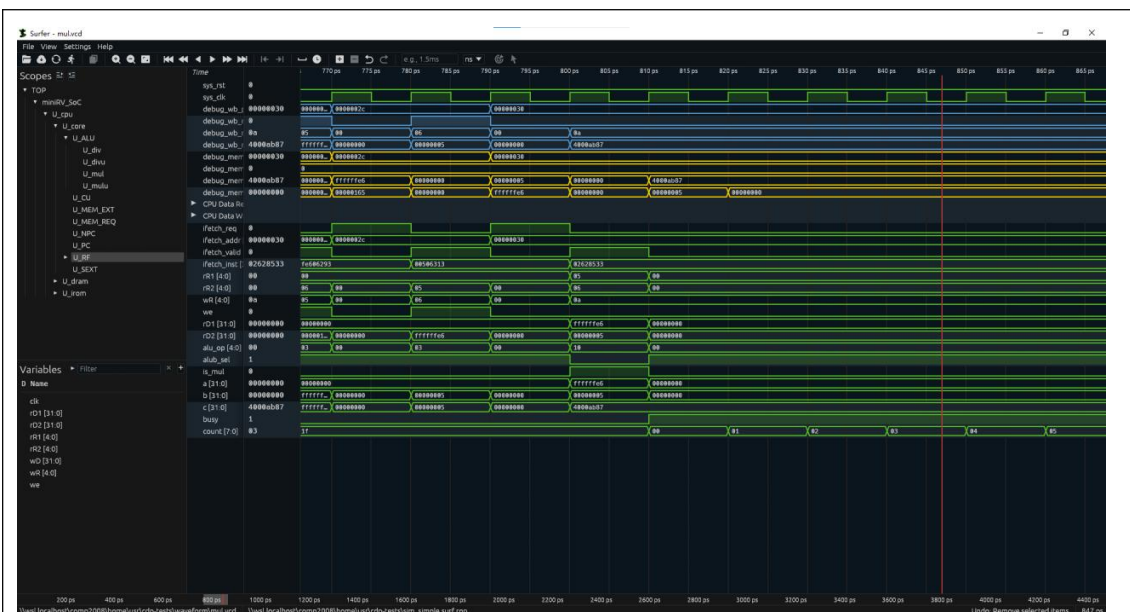
选取地址为 0x00000010 的指令进行分析, 其机器码为 0x00208023, 对应指令为 sb x2, 0(x1)。

在 280 ps 时, ifetch_req=1, ifetch_addr=0x00000010, 说明 CPU 向指令存储器发出取指请求。

在 290 ps 时, ifetch_valid=1, ifetch_inst=0x00208023, 指令取回并完成译码。此时基址寄存器 rf_rd1=0x00002000, 待写数据 rf_rd2=0xffffffff, 立即数扩展结果 ext=0x00000000。ALU 计算得到: $alu_c = 0x00002000 + 0x00000000 = 0x00002000$, 同时 ram_wop=1, 表示当前执行字节写操作。

在 300 ps 时可以看出, daccess_wen=4'b0001, daccess_addr=0x00002000, daccess_wdata=0xffffffff, 说明 CPU 向地址 0x2000 发出写请求。由于写使能为 0001, 因此只写入最低字节, 即将 0xaa 写入存储器。此时 debug_mem_we=0001、debug_mem_waddr=0x00002000、debug_mem_wdata=0xffffffff, 与实际访存信号一致。

在 310 ps 时, daccess_wresp=1, 表示数据存储器已经完成写操作。到 320 ps, ifetch_addr 更新为 0x00000014, CPU 开始读取下一条指令, 说明该条 sb 指令执行结束。仿真结果符合 sb 指令按字节写入存储器的预期功能。

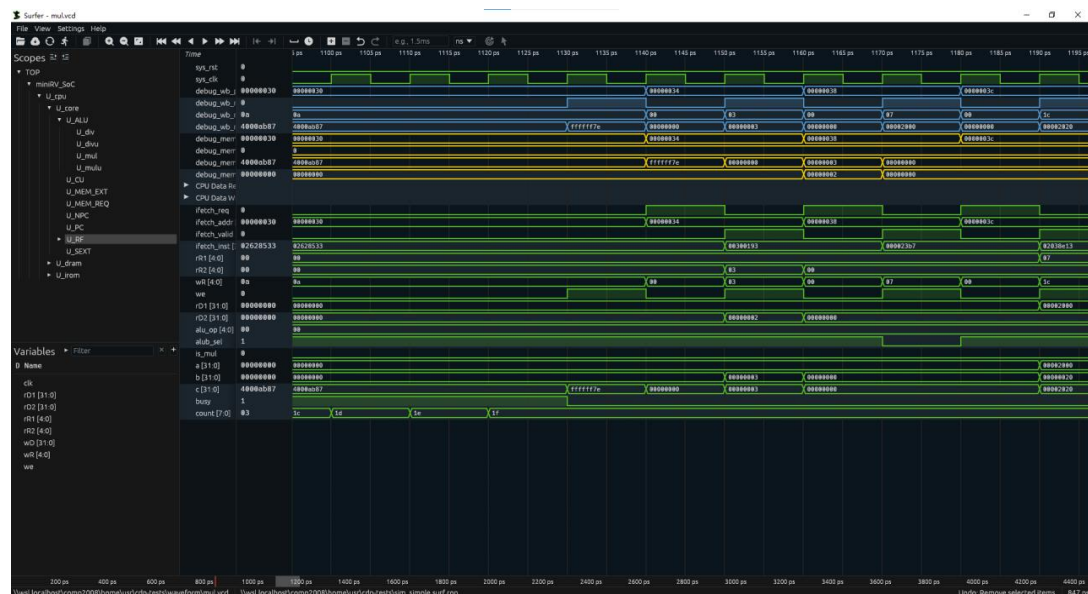


mul 指令仿真分析:

选取地址为 0x00000030 的指令进行分析, 其机器码为 0x02628533, 对应指令为 mul x10 x5 x6。

在 790 ps 时, ifetch_req=1, ifetch_addr=0x00000030, CPU 向指令存储器发出取指请求。

在 800 ps 时, ifetch_valid=1, ifetch_inst=0x02628533, 指令取回并完成译码。此时 rR1=05, rR2=06, wR=0a, 同时 rD1=ffffffe6, 即十进制数 -26, rD2=00000005, 即十进制数 5。alu_op=10, 对应于乘法操作, 同时 alub_sel 为 0, 说明 alu B 端的输入选择寄存器 rD2 的值。busy 信号处于高位说明正在进行多周期乘法操作, 并且计数器开始计数。

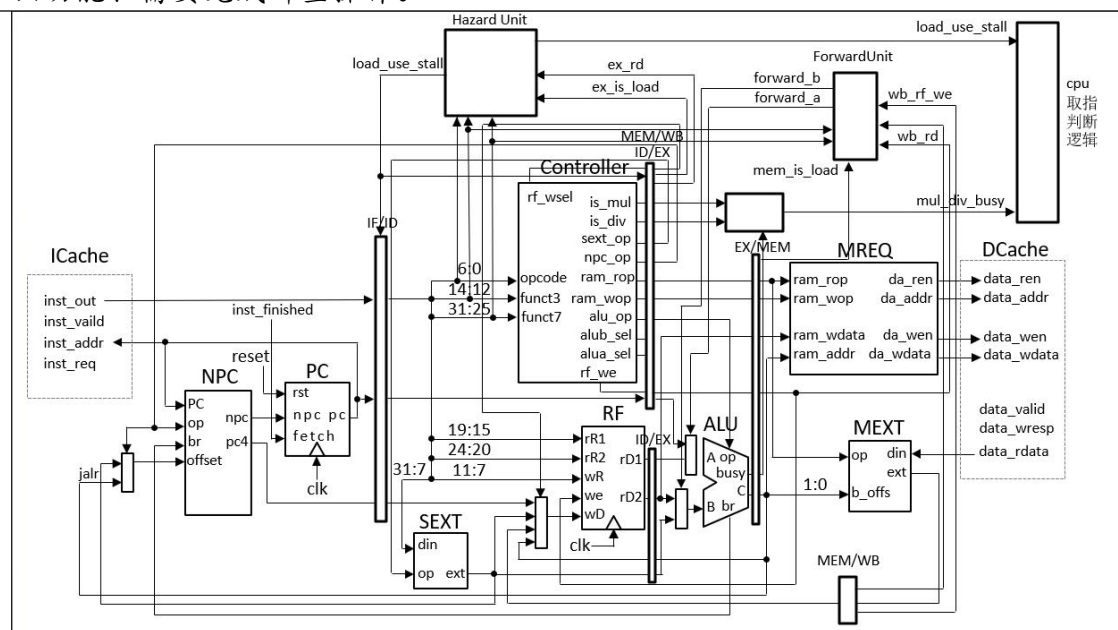


在 1130 ps 时, we=1, 表示将计算结果写回寄存器, 写回的结果是 c=ffffff7e, 对应十进制的 -130, 说明进行了正确的乘法操作, 计数器的值 count=1f, 说明乘法指令进行了 32 拍, 同时 busy 信号置零, 多周期乘法指令结束。

2 流水线 CPU 及 SoC 设计与实现

2.1 流水线 CPU 数据通路

要求：绘制完整的流水线数据通路图，无需画出模块内的具体逻辑，但要标出模块的接口信号名、模块之间信号线的信号名和位宽，并用文字阐述各模块的功能。数据通路图应当能体现出流水线是如何划分的，并用文字阐述每个流水级具备什么功能、需要完成哪些操作。



1. ICache (指令缓存)

用于缓存最近访问的指令，降低指令访存延迟。根据 PC 地址在标签阵列中查找：命中时一个周期返回指令；未命中时通过 AXI 发起突发读填充缓存行后返回，有效提高取指效率。

2. PC (程序计数器)

用于保存当前指令地址，并向指令存储器发出取指请求。每条指令执行完成后，PC 更新为 NPC 模块计算出的下一条指令地址。

3. NPC (下一 PC 生成模块)

根据当前 PC、立即数偏移量、分支判断结果和控制信号，计算下一条指令地址。普通指令选择 PC+4，跳转或分支指令选择相应的目标地址。

4. Controller (控制器)

对指令中的 opcode、funct3 和 funct7 字段进行译码，判断当前指令类型，并产生寄存器写使能、ALU 操作、立即数扩展、访存方式和下一 PC 选择等控制信号。

5. RF (寄存器堆)

用于保存通用寄存器数据。根据指令中的源寄存器编号同时读出两个操作数，并在写使能有效时将运算结果或访存数据写入目标寄存器。

6. SEXT (立即数扩展模块)

根据不同指令格式提取指令中的立即数字段，并将其进行符号扩展或重新拼接，生成 32 位立即数供 ALU、访存和跳转计算使用。

7. ALU (算术逻辑单元)

完成加、减、移位、逻辑运算和比较等操作。对于访存指令，ALU 还负责计算

访存地址；对于分支指令，ALU 输出分支判断结果。

8. 乘除法单元

用于完成乘法和除法相关指令。运算过程中通过 `mul_div_busy` 信号表示当前单元是否仍在工作，避免 CPU 提前更新执行状态。

9. MREQ (数据访存请求模块)

将 CPU 内部的访存操作转换为数据存储器接口信号，生成读使能、写使能、访存地址和写入数据，并接收数据存储器返回的有效信号或写响应信号。

10. MEXT (访存数据扩展模块)

对 Load 指令从存储器读回的数据进行处理。根据访问类型和地址低位，选取对应的字节、半字或字，并完成符号扩展或零扩展。

11. DCache (数据缓存)

用于缓存最近访问的数据，降低数据访存延迟。读命中时一个周期返回数据，读未命中时填充缓存行后返回。写操作采用写直达策略，外设地址自动绕过缓存实现直通访问。

12. 多路选择器

根据控制信号在多组输入数据中选择正确的数据通路，主要用于选择下一 PC、ALU 第二操作数以及寄存器写回数据等。

13. Hazard Unit (冒险检测单元)

用于检测流水线中的 Load-Use 冲突。在译码阶段检查当前 ID 指令的源操作数是否依赖 EX 阶段正在执行的 Load 指令的目标寄存器。若检测到冒险则产生 stall 信号，暂停 PC 和 IF/ID 寄存器并插入气泡。

14. Forward Unit (前递单元)

用于解决流水线中的数据依赖。在执行阶段检查当前 EX 指令的源操作数是否与 MEM/WB 阶段即将写回的目标寄存器匹配。若匹配则产生 forward 选择信号，控制 ALU 输入 MUX 从 EX/MEM 或 MEM/WB 寄存器提前获取数据。

15. CPU 取指判断逻辑

用于控制取指请求的发起时机。综合判断流水线是否阻塞、有无 Load-Use 冒险、指令队列是否已满、是否在进行乘除法指令，决定 pc 是否向指令存储器发起取指请求。

2.2 流水线 SoC 详细设计

要求：(1) 结合关键代码详细说明流水线冒险的检测及解决方法。(2) 绘制总线模块的状态转换图，标注清楚状态转移条件及每个状态产生的输出信号；结合关键代码详细说明总线及外设 I/O 接口电路的实现逻辑。

2.2.1 流水线冒险的检测与解决

(1) 普通 RAW 数据冒险与数据前递

当前指令进入 EX 阶段时，ForwardUnit 比较其源寄存器编号与 EX/MEM、MEM/WB 阶段的目的寄存器编号。当寄存器编号相同，并且前方指令需要写回寄存器时，将前方指令的结果直接送到当前指令的 ALU 输入，而不必等待结果真正写回寄存器堆。

代码中，forward_a 和 forward_b 的取值含义为：

选择信号	数据来源
2'b00	ID/EX 流水寄存器中保存的原始操作数
2'b10	EX/MEM 阶段的运算结果
2'b01	MEM/WB 阶段即将写回的数据

相关核心代码如下：

```
always @(*) begin
    forward_a = 2'b00;
    forward_b = 2'b00;

    if (mem_valid && mem_rf_we && !mem_is_load &&
        (mem_rd != 5'd0) && (mem_rd == ex_rs1))
        forward_a = 2'b10;
    else if (wb_valid && wb_rf_we &&
        (wb_rd != 5'd0) && (wb_rd == ex_rs1))
        forward_a = 2'b01;

    if (mem_valid && mem_rf_we && !mem_is_load &&
        (mem_rd != 5'd0) && (mem_rd == ex_rs2))
        forward_b = 2'b10;
    else if (wb_valid && wb_rf_we &&
        (wb_rd != 5'd0) && (wb_rd == ex_rs2))
        forward_b = 2'b01;
end
```

EX/MEM 阶段的前递优先级高于 MEM/WB 阶段，因为 EX/MEM 中的结果对应距离当前指令最近的前一条指令。Load 指令的数据在 EX/MEM 阶段尚未从存储器返回，因此代码通过 !mem_is_load 禁止将 Load 指令的地址计算结果误当前递数据。

在 cpu_core.v 中，根据前递选择信号确定 ALU 实际使用的操作数：


```

always @(*) begin
    case (forward_a_sel)
        2'b10: ex_rs1_forwarded = exmem_wb_predata_q;
        2'b01: ex_rs1_forwarded = memwb_wdata_q;
        default: ex_rs1_forwarded = idex_rs1_data_q;
    endcase

    case (forward_b_sel)
        2'b10: ex_rs2_forwarded = exmem_wb_predata_q;
        2'b01: ex_rs2_forwarded = memwb_wdata_q;
        default: ex_rs2_forwarded = idex_rs2_data_q;
    endcase
end

```

同样的选择逻辑也应用于第二个源操作数，因此既能处理 ALU 指令之间的数据相关，也能处理 Store 指令写数据对前方计算结果的依赖。数据前递可以避免普通 RAW 冒险引起的不必要暂停。

(2) Load-Use 冒险与流水线暂停

Load 指令只有在 MEM 阶段收到存储器返回的数据后，结果才有效。因此，当 EX 阶段是一条 Load 指令，而 ID 阶段的后一条指令立即使用 Load 的目的寄存器时，单纯依靠 EX/MEM 前递无法解决。

HazardUnit 首先根据 ID 阶段指令的 opcode 判断该指令是否真正使用 rs1 和 rs2，再比较这些源寄存器与 EX 阶段 Load 指令的目的寄存器。冒险检测条件为：

```

assign load_use_stall = id_valid && ex_valid && ex_is_load &&
    (ex_rd != 5'd0) &&
    ((id_use_rs1 && (id_rs1 == ex_rd)) ||
    (id_use_rs2 && (id_rs2 == ex_rd)));

```

当 load_use_stall 有效时：停止发出新的取指请求、保留指令队列头部的 ID 阶段指令、允许前面的 Load 指令继续向 MEM 阶段执行、将 idex_valid_q 清零，在 ID/EX 之间插入一个气泡。

```

else if (load_use_stall) begin
    idex_valid_q <= 1'b0;
end

```

经过一个暂停周期后，Load 数据进入 MEM/WB 阶段，后一条指令便可以通过 MEM/WB 前递获得正确数据。

(3) 访存等待与乘除法等待

Cache 或主存不能保证在发出请求的同一周期返回结果。因此，当 MEM 阶段正在执行 Load 或 Store，但尚未收到 daccess_rvalid 或 daccess_wresp 时，产生 mem_stage_stall：

```

assign mem_stage_stall = exmem_valid_q && mem_has_operation &&
    mem_requestable && !mem_response;

```


乘除法使用迭代运算模块。当乘除法单元仍处于忙状态时，产生 `muldiv_wait`。两种等待信号合并为全局保持信号：

```
wire pipeline_hold = mem_stage_stall | muldiv_wait;
```

当 `pipeline_hold` 有效时，CPU 停止继续取指，并保持当前 ID/EX 及相关流水线状态，直至访存响应返回或乘除法运算结束。代码还使用 `mem_req_sent_q` 和 `muldiv_started_q` 记录请求是否已经发出，避免流水线暂停期间重复发送访存请求或重复启动乘除法运算。

（4）控制冒险处理

本设计采用静态预测不跳转的方法。正常情况下，取指地址按照 PC+4 顺序更新。分支和跳转指令进入 EX 阶段后，CPU 根据 ALU 产生的分支条件判断是否真正跳转：

```
wire branch_redirect = ex_stage_done && branch_or_jump_taken;
```

当分支成立，或者执行 JAL、JALR 指令时，`branch_redirect` 有效，PC 被修改为正确的跳转目标地址。同时执行清空双入口取指队列、将 `idex_valid_q` 清零、丢弃已经发出但尚未返回的错误路径取指响应、从正确目标地址重新取指。

```
else if (branch_redirect) begin
    ifq_count_q <= 2'd0;
end

else if (branch_redirect) begin
    idex_valid_q <= 1'b0;
end
```

2.2.2 AXI 总线控制器状态机设计

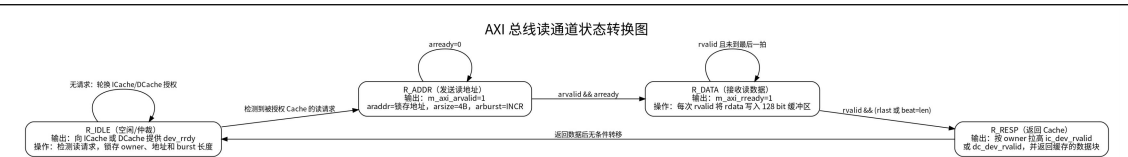
本设计的 `axi_master.v` 分别使用一个读状态机和一个写状态机。由于 AXI 读通道和写通道相互独立，两套状态机可以分别管理读、写请求。AXI 传输遵循 ready-valid 握手规则，只有 `ready` 和 `valid` 同时有效时，当前地址或数据才被成功传输。

（1）读状态机

读状态机包含四个状态：

状态	主要输出	功能
R_IDLE	ic_dev_rrdy 或 dc_dev_rrdy	在 ICache 和 DCache 之间进行仲裁并接收读请求
R_ADDR	m_axi_arvalid=1	通过 AR 通道发送读地址和猝发参数
R_DATA	m_axi_rready=1	通过 R 通道接收数据并存入 128 位缓冲区
R_RESP	ic_dev_rvalid 或 dc_dev_rvalid	将完整的数据块返回给提出请求的 Cache

状态转移过程如下：



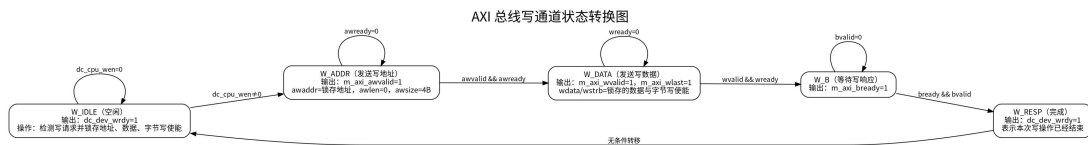
写地址和写数据分别通过 AW 通道与 W 通道发送。当两部分均被从设备接收后，状态机进入 W_B 等待写响应，收到 bvalid 后才认为本次写操作完成。

(2) 写状态机

写状态机包含五个状态：

状态	主要输出	功能
W_IDLE	dc_dev_wrdy=1	接收 DCache 写请求并锁存地址、数据和字节写使能
W_ADDR	m_axi_awvalid=1	通过 AW 通道发送写地址
W_DATA	m_axi_wvalid=1、 m_axi_wlast=1	通过 W 通道发送写数据
W_B	m_axi_bready=1	等待从设备返回写响应
W_RESP	dc_dev_wrdy=1	表示写操作处理结束，然后返回空闲状态

状态转移过程如下：



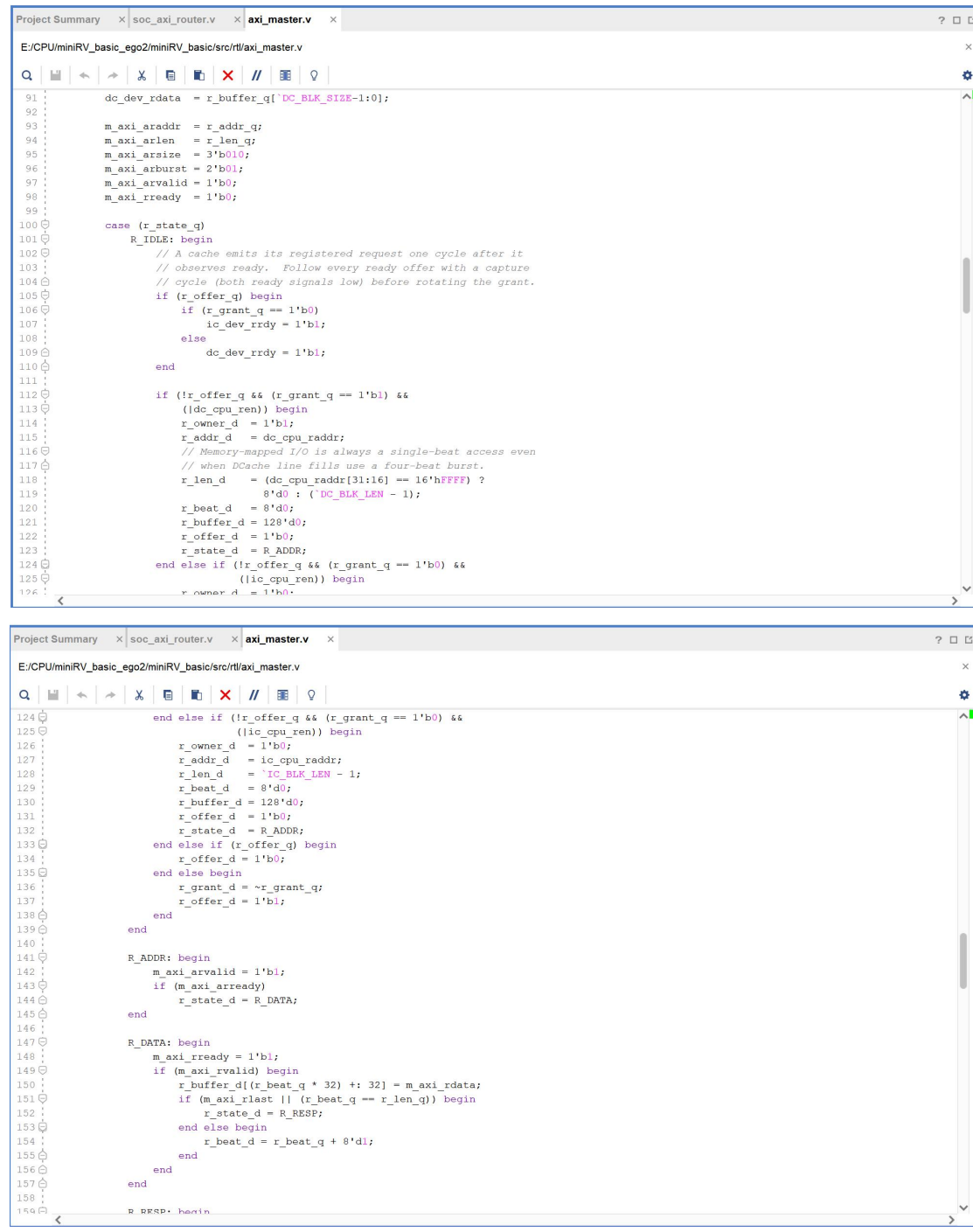
主要代码截图：

读状态机

```

Project Summary | soc_axi_router.v | axi_master.v |
E:/CPU/miniRV_basic_ego2/miniRV_basic/src/rtl/axi_master.v

58 //*****
59 * Read channel
60 *****
61 //*****
62 localparam R_IDLE = 2'd0;
63 localparam R_ADDR = 2'd1;
64 localparam R_DATA = 2'd2;
65 localparam R_RESP = 2'd3;
66
67 reg [1:0] r_state_q, r_state_d;
68 reg r_grant_q, r_grant_d;
69 reg r_offer_q, r_offer_d;
70 reg r_owner_q, r_owner_d; // 0: ICACHE, 1: DCache
71 reg [31:0] r_addr_q, r_addr_d;
72 reg [ 7:0] r_len_q, r_len_d;
73 reg [ 7:0] r_beat_q, r_beat_d;
74 reg [127:0] r_buffer_q, r_buffer_d;
75
76 always @(*) begin
77     r_state_d = r_state_q;
78     r_grant_d = r_grant_q;
79     r_offer_d = r_offer_q;
80     r_owner_d = r_owner_q;
81     r_addr_d = r_addr_q;
82     r_len_d = r_len_q;
83     r_beat_d = r_beat_q;
84     r_buffer_d = r_buffer_q;
85
86     ic_dev_rrdy = 1'b0;
87     dc_dev_rrdy = 1'b0;
88     ic_dev_rvalid = 1'b0;
89     dc_dev_rvalid = 1'b0;
90     ic_dev_rdata = r_buffer_q[IC_BLK_SIZE-1:0];
91     dc_dev_rdata = r_buffer_q[DC_BLK_SIZE-1:0];
92
93     m_axi_araddr = r_addr_q;
  
```

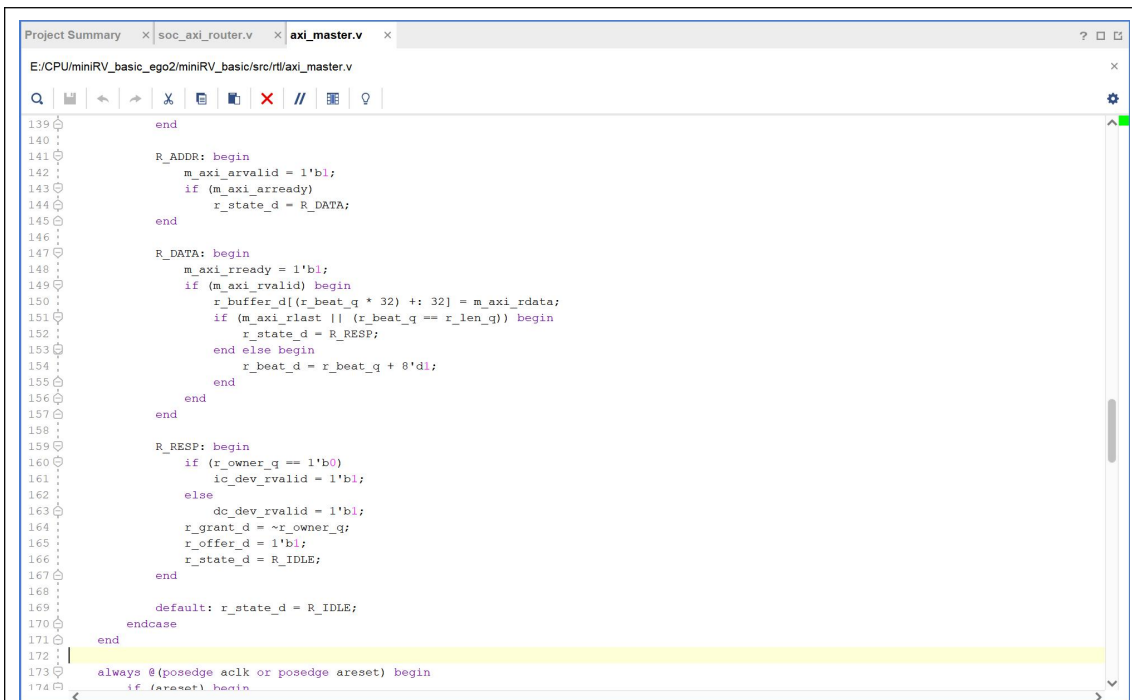


```

Project Summary x soc_axi_router.v x axi_master.v x
E:/CPU/miniRV_basic_ego2/miniRV_basic/src/rtl/axi_master.v

91      dc_dev_rdata = r_buffer_q['DC_BLK_SIZE-1:0];
92
93      m_axi_araddr = r_addr_q;
94      m_axi_arlen = r_len_q;
95      m_axi_arsize = 3'b010;
96      m_axi_arburst = 2'b01;
97      m_axi_arvalid = 1'b0;
98      m_axi_rready = 1'b0;
99
100     case (r_state_q)
101     R_IDLE: begin
102         // A cache emits its registered request one cycle after it
103         // observes ready. Follow every ready offer with a capture
104         // cycle (both ready signals low) before rotating the grant.
105         if (r_offer_q) begin
106             if (r_grant_q == 1'b0)
107                 ic_dev_rdy = 1'b1;
108             else
109                 dc_dev_rdy = 1'b1;
110         end
111
112         if (!r_offer_q && (r_grant_q == 1'b1) &&
113             (!dc_cpu_ren)) begin
114             r_owner_d = 1'b1;
115             r_addr_d = dc_cpu_raddr;
116             // Memory-mapped I/O is always a single-beat access even
117             // when DCache line fills use a four-beat burst.
118             r_len_d = (dc_cpu_raddr[31:16] == 16'hFFFF) ?
119                 8'd0 : ('DC_BLK_LEN - 1);
120             r_beat_d = 8'd0;
121             r_buffer_d = 128'd0;
122             r_offer_d = 1'b0;
123             r_state_d = R_ADDR;
124         end else if (!r_offer_q && (r_grant_q == 1'b0) &&
125             (!ic_cpu_ren)) begin
126             r_owner_d = 1'b0;
127             r_addr_d = ic_cpu_raddr;
128             r_len_d = 'IC_BLK_LEN - 1;
129             r_beat_d = 8'd0;
130             r_buffer_d = 128'd0;
131             r_offer_d = 1'b0;
132             r_state_d = R_ADDR;
133         end else if (r_offer_q) begin
134             r_offer_d = 1'b0;
135         end else begin
136             r_grant_d = ~r_grant_q;
137             r_offer_d = 1'b1;
138         end
139     end
140
141     R_ADDR: begin
142         m_axi_arvalid = 1'b1;
143         if (m_axi_arready)
144             r_state_d = R_DATA;
145     end
146
147     R_DATA: begin
148         m_axi_rready = 1'b1;
149         if (m_axi_rvalid) begin
150             r_buffer_d[(r_beat_q * 32) +: 32] = m_axi_rdata;
151             if (m_axi_rlast || (r_beat_q == r_len_q)) begin
152                 r_state_d = R_RESP;
153             end else begin
154                 r_beat_d = r_beat_q + 8'd1;
155             end
156         end
157     end
158
159     R_RESP: begin

```

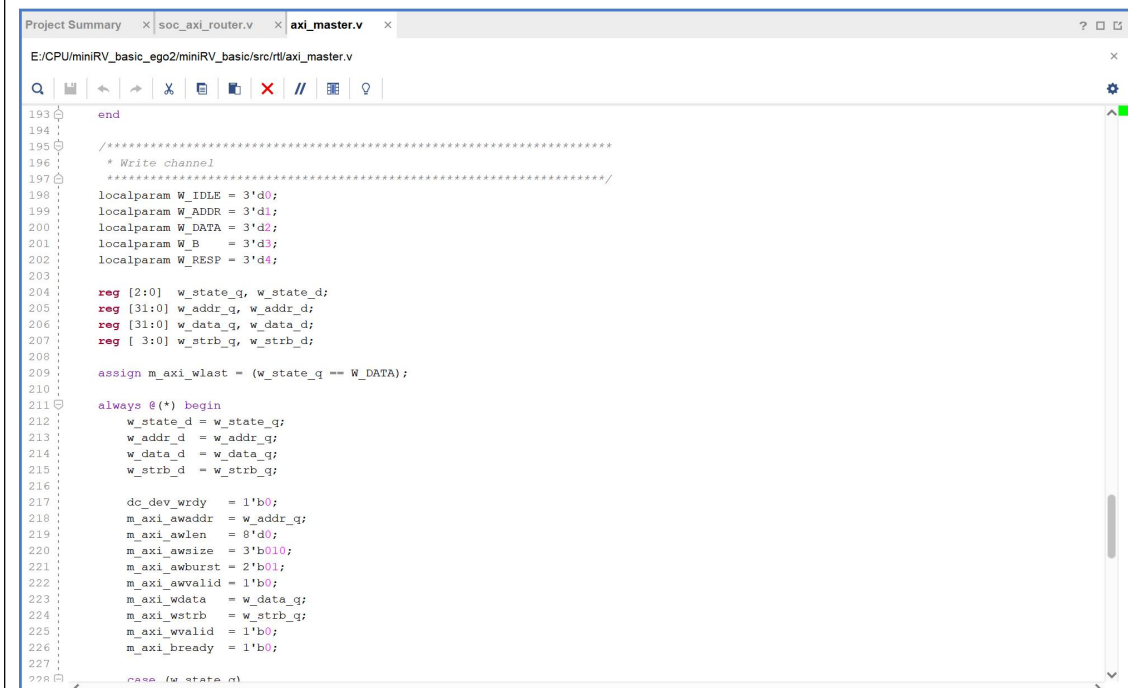


```

139:         end
140:
141:         R_ADDR: begin
142:             m_axi_arvalid = 1'b1;
143:             if (m_axi_arready)
144:                 r_state_d = R_DATA;
145:             end
146:
147:         R_DATA: begin
148:             m_axi_rready = 1'b1;
149:             if (m_axi_rvalid) begin
150:                 r_buffer_d[(r_beat_q * 32) +: 32] = m_axi_rdata;
151:                 if (m_axi_rlast || (r_beat_q == r_len_q)) begin
152:                     r_state_d = R_RESP;
153:                 end else begin
154:                     r_beat_d = r_beat_q + 8'd1;
155:                 end
156:             end
157:         end
158:
159:         R_RESP: begin
160:             if (r_owner_q == 1'b0)
161:                 ic_dev_rvalid = 1'b1;
162:             else
163:                 dc_dev_rvalid = 1'b1;
164:                 r_grant_d = ~r_owner_q;
165:                 r_offer_d = 1'b1;
166:                 r_state_d = R_IDLE;
167:             end
168:
169:         default: r_state_d = R_IDLE;
170:     endcase
171: end
172:
173: always @(posedge aclk or posedge areset) begin
174:     if (areset) begin

```

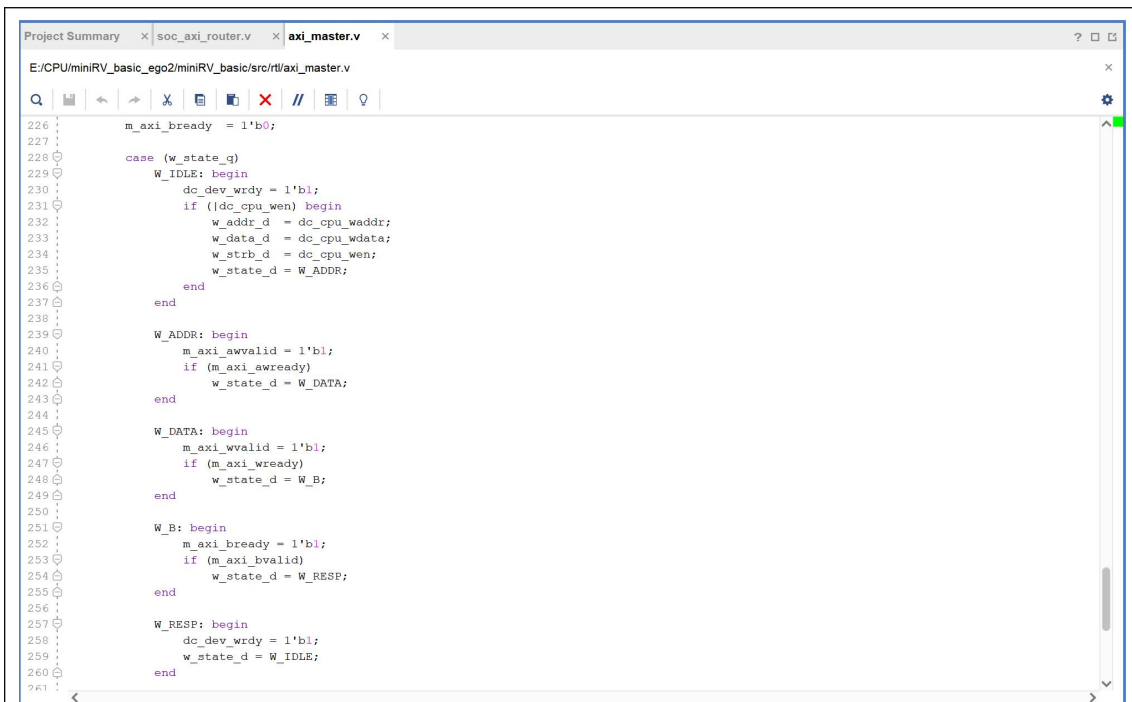
写状态机:



```

193:         end
194:
195:         /* Write channel */
196:         *****
197:         *****
198:         localparam W_IDLE = 3'd0;
199:         localparam W_ADDR = 3'd1;
200:         localparam W_DATA = 3'd2;
201:         localparam W_B = 3'd3;
202:         localparam W_RESP = 3'd4;
203:
204:         reg [2:0] w_state_q, w_state_d;
205:         reg [31:0] w_addr_q, w_addr_d;
206:         reg [31:0] w_data_q, w_data_d;
207:         reg [ 3:0] w_strb_q, w_strb_d;
208:
209:         assign m_axi_wlast = (w_state_q == W_DATA);
210:
211:         always @(*) begin
212:             w_state_d = w_state_q;
213:             w_addr_d = w_addr_q;
214:             w_data_d = w_data_q;
215:             w_strb_d = w_strb_q;
216:
217:             dc_dev_wrdy = 1'b0;
218:             m_axi_awaddr = w_addr_q;
219:             m_axi_awlen = 8'd0;
220:             m_axi_awsz = 3'b010;
221:             m_axi_awburst = 2'b01;
222:             m_axi_awvalid = 1'b0;
223:             m_axi_wdata = w_data_q;
224:             m_axi_wstrb = w_strb_q;
225:             m_axi_wvalid = 1'b0;
226:             m_axi_bready = 1'b0;
227:
228:             case (w_state_q)

```



2.2.3 总线地址路由与外设 I/O 接口

(1) 地址路由

在 soc_axi_router.v 中，通过地址高 16 位判断当前请求是否为外设访问：

```

wire aw_is_io = (s_axi_awaddr[31:16] == 16'hFFFF);
wire ar_is_io = (s_axi_araddr[31:16] == 16'hFFFF);

```

当地址不属于 0xFFFF_xxxx 时，请求直接转发给 BRAM 主存；当地址属于外设空间时，不再访问主存，而是产生 io_wr_en 或 io_rd_en，将地址、写数据和字节写使能送入 soc_peripherals。

对于外设写操作，路由模块先保存 AW 通道传来的写地址，再等待 W 通道写数据，完成后产生 AXI 写响应。对于外设读操作，路由模块读取 io_rd_data，并通过 rvalid 和 rlast 向 AXI 主设备返回单拍读数据。

(2) 外设地址分配

本设计支持拨码开关、LED、数码管、UART 和 64 位计时器五类外设，地址分配如下：

外设端口	地址	属性	功能
拨码开关	0xFFFF_0000	只读	读取板上开关状态
LED	0xFFFF_1000	读写	控制 16 位 LED
数码管	0xFFFF_2000	读写	设置 8 位数码管显示值
UART RX FIFO	0xFFFF_3000	只读	读取接收到的字符
UART TX FIFO	0xFFFF_3004	只写	写入需要发送的字符
UART 状态	0xFFFF_3008	只读	查询收发 FIFO 状态
UART 控制	0xFFFF_300C	只写	清空或控制收发 FIFO
计时器低 32 位	0xFFFF_4000	只读	读取 64 位计时器低 32 位

计时器高 32 位	0xFFFF_4008	只读	读取 64 位计时器高 32 位
-----------	-------------	----	------------------

(3) 各外设实现逻辑

拨码开关属于板外异步输入，代码使用两级寄存器同步后再提供给 CPU，减少异步信号直接进入系统时钟域造成的不稳定。

LED 和数码管使用寄存器保存 CPU 写入的数据，并根据 io_wr_strb 按字节更新：

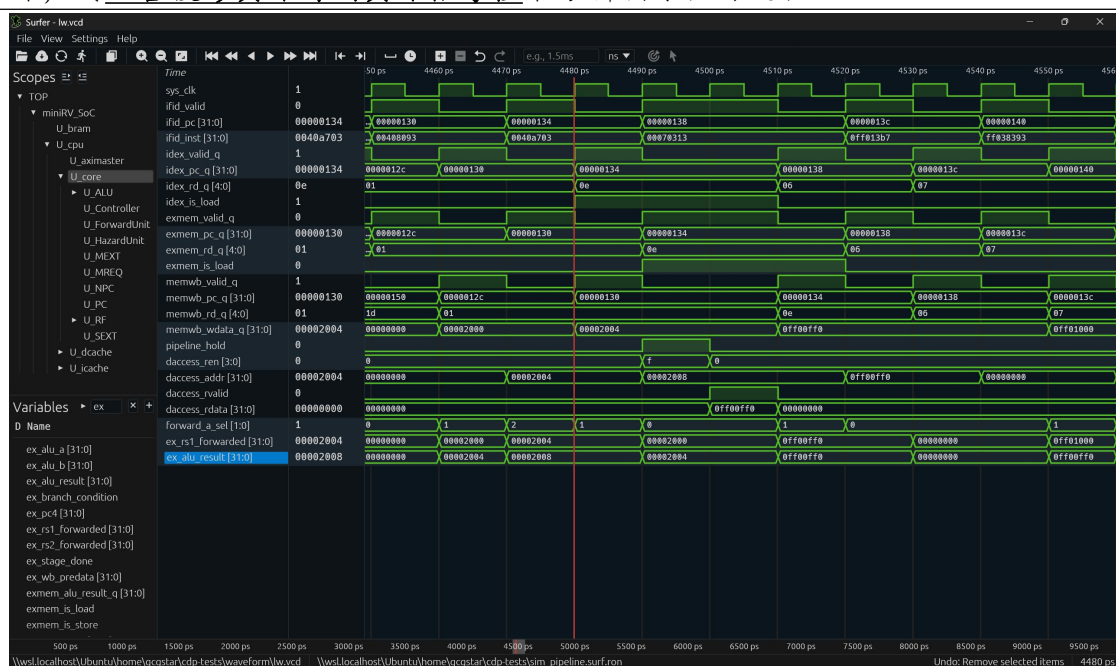
```
if (io_wr_en && io_wr_addr[15:0] == 16'h1000) begin
    if (io_wr_strb[0]) led_q[7:0] <= io_wr_data[7:0];
    if (io_wr_strb[1]) led_q[15:8] <= io_wr_data[15:8];
end
```

数码管模块使用扫描计数器依次选择不同数码管位，并通过 seven_seg 函数把 4 位十六进制数转换为对应段码。计时器采用 64 位寄存器，每个系统时钟周期加 1。CPU 可分别读取其高 32 位和低 32 位。UART 按照 115200 波特率工作，内部设置 16 字节的发送 FIFO 和接收 FIFO。CPU 向 0xFFFF_3004 写入字符后，发送模块依次输出起始位、8 位数据和停止位；接收模块对 rx 信号同步并按照波特率采样，将接收到的字符写入 RX FIFO。

2.3 流水线 SoC 仿真分析

要求：(1) 结合波形截图分析流水线 CPU 处理数据冒险的过程（若仅实现了理想流水，则结合波形截图分析理想流水执行指令的过程）。(2) 结合波形截图分析从 Cache 发生缺失并向总线发出访存请求，到总线读取主存并返回数据给

Cache 的过程。截图需清晰，且需要包含信号名和指令执行所需的关键信号；此外，要结合波形具体时刻具体信号值来分析指令执行过程。



选取以下连续三条指令进行分析：

```
130: 00408093    addi x1, x1, 4
134: 0040a703    lw     x14, 4(x1)
138: 00070313    addi x6, x14, 0
```

其中，lw 需要使用前一条 addi 更新后的 x1 作为基址，后一条 addi 又需要使用 lw 加载到 x14 中的数据，因此存在连续的 RAW 数据冒险。

在 4480 ps 时，`idex_pc_q=0x00000134`、`idex_rd_q=0x0e`、`idex_is_load=1`，说明 `lw x14,4(x1)` 正在执行阶段。此时前一条 `addi` 的结果已经进入 MEM/WB 阶段，`memwb_wdata_q=0x00002004`。由于 lw 需要使用更新后的 x1，控制信号 `forward_a_sel=1`，前递后的操作数 `ex_rs1_forwarded=0x00002004`，ALU 计算结果为： $0x00002004 + 0x00000004 = 0x00002008$ 。因此，前一条 `addi` 与 `lw` 之间的数据冒险通过 MEM/WB 前递得到解决。

在 4490 ps 时，lw 进入访存阶段，`exmem_pc_q=0x00000134`、`exmem_is_load=1`。此时 `daccess_ren=4'hF`、`daccess_addr=0x00002008`，CPU 向数据存储器发出读请求。由于 Load 数据尚未返回，`pipeline_hold=1`，流水线暂时保持。波形中 `ifid_pc` 保持为 `0x00000138`，`idex_pc_q` 仍保持为 `0x00000134`，避免后一条 `addi x6,x14,0` 提前使用尚未产生的 x14 数据。

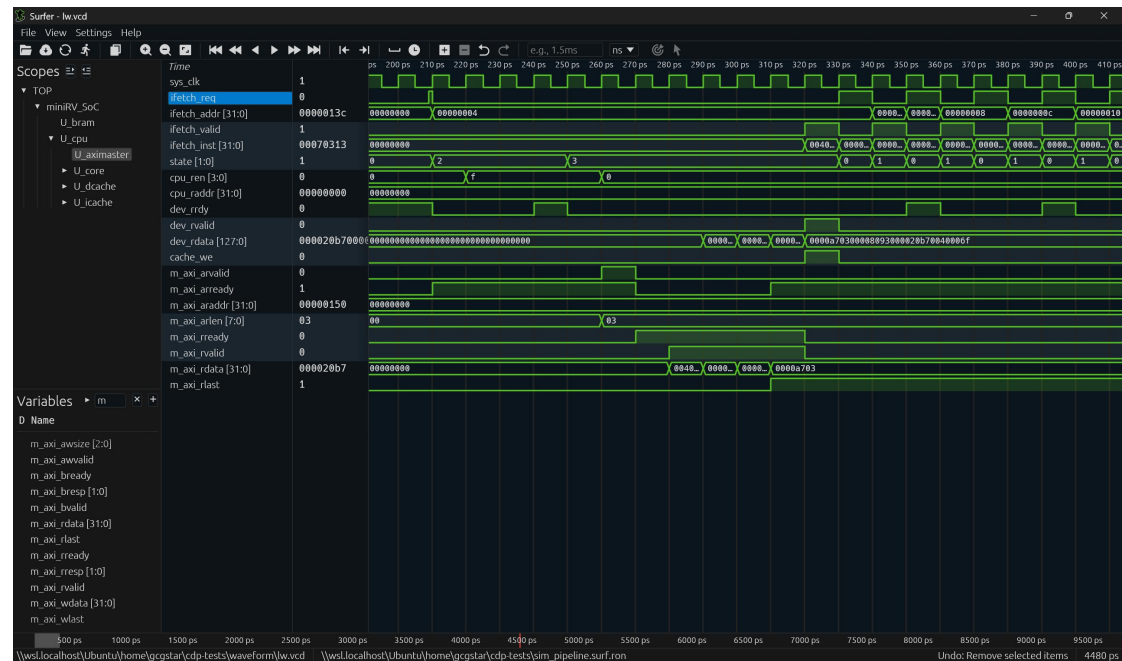
在 4500 ps 时，`daccess_rvalid=1`，存储器返回数据：`daccess_rdata = 0x0ff0ff0`。数据返回后，访存等待结束，流水线可以继续向后执行。

在 4510 ps 时，lw 进入 MEM/WB 阶段，`memwb_pc_q=0x00000134`、`memwb_rd_q=0x0e`、`memwb_wdata_q=0x0ff0ff0`，说明 Load 结果即将写入 x14。同时，后一条 `addi` 进入执行阶段，`idex_pc_q=0x00000138`。此时 `forward_a_sel=1`，`ex_rs1_forwarded=0x0ff0ff0`，说明 `addi` 直接从 MEM/WB 阶段取得 lw 的结果，而不是读取寄存器堆中的旧值。由于该指令的立即数为 0，ALU 结果为：`ex_alu_result = 0x0ff0ff0`。

在 4530 ps 时，`memwb_pc_q=0x00000138`、`memwb_rd_q=0x06`，说明 `addi`

x6,x14,0 的结果已经传递到写回阶段，并正确写入 x6。

综上，该流水线 CPU 首先通过 pipeline_hold 等待 Load 数据返回，然后利用 MEM/WB 前递将数据传给相关指令，从而正确解决了 lw 与后一条 addi 之间的 Load-use 数据冒险。



选取复位后第一次取指过程进行分析。由于此时 ICache 中还没有对应指令块，因此会发生一次 ICache 缺失 (miss)，随后 ICache 需要通过总线控制器向主存发出读请求，并在主存返回数据后完成 Cache 填充。

在约 210 ps 时，CPU 已经发出取指请求，ifetch_addr=0x00000000，同时 ICache 的 state 由初始状态转入缺失处理状态，说明当前访问在 ICache 中未命中，需要从主存中取回指令块。

在 220 ps 左右，ICache 向总线控制器发出读请求，此时 cpu_ren=4'hf，cpu_raddr=0x00000000。其中 cpu_ren=4'hf 表示当前存在一次有效的整块读请求，读地址为 0x00000000。随后总线控制器接收该请求，并准备通过 AXI 读地址通道向主存发起访存。

在 260 ps 左右，AXI 读地址请求发出，可以看到 m_axi_arvalid=1，同时 m_axi_araddr=0x00000000，m_axi_arlen=8'h03。这说明总线控制器希望从主存中读取从地址 0x00000000 开始的一个 Cache 块。由于 arlen=3，按照 AXI 协议表示本次传输共需要返回 4 个 32 位数据，也就是正好组成一个 128 位 Cache 块。

在 280 ps~310 ps 之间，主存通过 AXI 读数据通道连续返回数据，表现为 m_axi_rvalid=1，同时 m_axi_rdata 依次给出 4 个 32 位字。波形中可以看到返回的数据依次组成了这一块指令内容，最后拼接成 dev_rdata = 128'h0000a703_00008093_000020b7_0040006f。

在最后一个数据返回时，m_axi_rlast=1，表示当前拍返回的是本次突发读传输的最后一个数据包，说明主存端的数据块已经全部返回完毕。

在 320 ps 左右，总线控制器将主存返回的整块数据送回 ICache，此时 dev_rvalid=1，dev_rdata 为完整的 128 位数据块，同时 cache_we=1，说明 ICache 对该数据块执行了填充写入操作。也就是说，这一拍中 ICache 已经把从主存读

回的数据写入了自己的数据阵列中，完成了 Cache refill。

Cache 填充完成后，CPU 就可以重新从 ICache 中取得所需指令。波形后续可以看到 ifetch_valid 重新有效，ifetch_inst 输出对应指令数据，说明这次“Cache 缺失→总线请求→主存返回→Cache 填充”的过程已经完成，CPU 恢复正常取指执行。

综上，从该波形可以看出：当 ICache 未命中时，先由 ICache 通过 cpu_ren 和 cpu_raddr 向总线控制器发出读请求；随后总线控制器通过 AXI 的 AR 通道向主存发送地址请求；主存再通过 R 通道分 4 拍返回数据，并在 rlast=1 时完成一次突发读；最后总线控制器把完整的 128 位数据块通过 dev_rvalid 和 dev_rdata 返回给 ICache，由 cache_we 完成填充写入。整个过程与 Cache 缺失处理机制一致。

3 设计过程中遇到的问题及解决方法

要求：(1) 设计过程中遇到的**有价值的错误**，或测试过程中遇到的**有价值的问题**。所谓有价值，指的是解决该错误或问题后，能够学到新的知识和技巧，或加深对已有知识的理解和运用。(2) 组内讨论解决了哪些问题，至少列出 2 个。

(1) 设计与测试中遇到的有价值问题

① 字节和半字访存处理错误。

在实现 LB、LH、SB、SH 等指令时，最开始没有正确处理地址低两位、符号扩展和写使能，导致访存结果错误。通过查看波形并修改数据选择逻辑，我们进一步理解了小端存储，以及字节、半字和字访存的区别。

② Trace 测试报错后不容易定位问题。

测试失败时，只能看到某个 PC 对应的结果不一致，无法直接判断是哪部分代码出错。后来我们通过 PC 查找反汇编指令，再结合波形检查控制信号、ALU 结果和写回数据，逐步定位错误，也掌握了利用反汇编和波形联合调试的方法。

③ ICache 与 DCache 冲突时的选择策略

在实现 AXI Master 读通道仲裁时，最开始采用 DCache 优先的固定优先级策略，有时会导致 ICache 的取指请求阻塞，流水线因取指停顿而空转，导致性能下降。通过查看波形并修改为轮转仲裁策略，进一步理解了多设备共享总线时公平性与优先级的权衡问题，以及取指阻塞对整个流水线性能的影响。

(2) 组内讨论解决的问题

① 解决了 A、B 组指令代码整合问题。

两部分代码合并后，出现了译码条件重复和控制信号取值不统一的问题。组内重新核对各条指令的编码和控制信号，并逐条进行 Trace 测试，最终完成了完整单周期 CPU 的整合。

② 解决了流水线冒险的处理问题。

一开始我们对哪些情况使用前递、暂停或清空流水线理解不一致。经过讨论和分析波形，最终确定普通数据相关使用前递，Load-use 冒险暂停一个周期，分支跳转时清空错误进入流水线的指令。

③ 解决了流水线阻塞信号相互影响导致的问题

流水线中存在访存等待、乘除运算等待和 Load-use 冒险等多种阻塞源，最初各阻塞信号独立控制流水线，导致多个阻塞同时触发时出现死锁或指令丢失。组内联合分析了各阻塞源的产生条件和作用范围，最终设计了 `inst_finished` 作为唯一的判断规则，并统一了各阻塞信号在流水线寄存器更新和取指控制中的使用方式，确保多源阻塞同时发生时流水线能够正确停顿和恢复。

4 总结

要求：(1) 学完本课程后的个人收获；(2) 对本课程的建议和意见。请在认真总结和思考后填写。

通过本课程的学习，我对单周期 CPU、五级流水线以及 SoC 的基本结构有了更清楚的认识，也理解了指令在 CPU 中的取指、译码、执行、访存和写回过程，掌握了 CPU 基本的内部结构以及各个功能模块之间的交流。在实际设计和调试中，我掌握了使用 Vivado 进行仿真、查看波形、运行 Trace 测试和下板验证的方法，同时也体会到硬件设计需要认真分析时序和控制信号，遇到问题时要结合代码和波形逐步定位。

我认为课程内容比较充实，但整体难度较大，前期上手较慢。建议增加完整工程的演示和常见错误的讲解，让同学先了解整体设计流程，再完成各部分任务，这样会更容易理解和实践。

5 AI 大模型使用日志

要求：总结实验过程中使用 AI 大模型帮助你解决了哪些主要问题，分别给出提问的提示词并提炼 AI 大模型回答的主要内容。

主要问题	提问的提示词	AI 回答的主要内容
理解 CPU 和流水线结构	“请用简单的语言解释单周期 CPU 和 IF、ID、EX、MEM、WB 五级流水线分别在做什么。”	解释了取指、译码、执行、访存和写回的过程，以及各级流水寄存器需要保存的数据和控制信号。
分析乘法器设计原理	“除了 booth 算法设计乘法器之外还有没有什么其他方式，有没有什么要注意的重点事项”	给出了移位乘法器的基本原理，并指出要注意位宽和对无符号数的处理
分析流水线冒险	“请分析 addi→lw 和 lw→addi 之间的数据冒险，说明什么时候前递，什么时候暂停。”	说明普通数据相关可通过 EX/MEM 或 MEM/WB 前递解决；Load-Use 冒险需要暂停流水线并插入气泡。
分析仿真波形	“结合波形中的 PC、指令、前递和写回信号，说明指令在各流水级的执行过程。”	帮助确定指令处于哪个流水阶段，并根据具体时刻和信号值分析前递、暂停及写回过程。
定位 Trace 和下板问题	“Basic Trace 和 AXI Trace 有什么区别？出现 Difference 或 COE 无法加载时应该怎样排查？”	说明了两类 Trace 的测试范围，并给出通过报错 PC、反汇编文件、VCD 波形和 Vivado IP 配置定位问题的方法。
整理 SoC 设计和实验报告	“请根据实际代码说明 AXI 状态机、外设地址映射和流水线冒险处理，并整理成实验报告语言。”	提炼了 AXI 读写状态、总线地址路由、UART 等外设实现逻辑，并协助将实验过程整理为较规范的报告内容。